JC1503 Object-Oriented Programming

Group Project — Personal Data System

Group size: 4–6 students

Programming Language: Python

Cours Coordinator: Ruizhe Li; Shahzad Mumtaz; Zhenhua Yu;

1. Instructions

The goal of this group project is to define and deliver your team's current best version of a Python personal software system, and to show that you can build it in a way that reflects the core ideas taught in JC1503. This is not just about getting something to run. It is about demonstrating that you can design a program as a software system with a clear structure, clear responsibilities across classes, and sensible choices of data structures, so that the system could realistically be extended or maintained later.

Your final submission should present a complete and coherent application in a chosen domain (for example, a record-management system, tracking system, booking system, or another realistic scenario). The system should do more than a single task. It should support a small set of meaningful operations (create, update, delete, search/query, and safe error handling), and it should feel like an application rather than a collection of disconnected scripts.

Wherever possible, you should justify your design decisions with evidence from your own development work. For example, you can explain why you modelled the problem using particular classes, why you chose certain relationships (inheritance vs composition), and why a specific data structure is appropriate for a given operation. You should also use testing and debugging evidence to support your decisions, what you tested, what broke, how you found the problem, and how you fixed it. The point is to show that the system did not appear instantly; it was developed, refined, and improved through iterations.

A key expectation is that your system creates its working environment and data in a realistic way. At first run, the program should initialize a dataset and store it to file, and from then onwards it should load and update that data rather than recreating everything each time. This means your application behaves like a real system: it has state, it evolves, and it can be used repeatedly.

The primary deliverable is a functioning software system. The written report and videos explain and justify how that system was designed and developed.

This is a **team submission**. One member should submit on behalf of the whole group. Please make sure all names and contributions appear clearly in the report, and that you submit **everything together in a single zip file**, as this makes it easier to track your work. You must also include the required contribution information (who did what) as well in the process presentation component, so that individual responsibilities are clear.

2. Scenario

Each group should select a single application domain and develop a coherent software system around it. The chosen scenario should represent a realistic context in which records are created, modified, searched, and maintained over time.

Suitable examples include the following examples:

- student course management system,
- library borrowing system,
- hospital patient records,
- personal finance tracker,
- game inventory or trading system,
- fitness training tracker,
- pet management system,
- equipment maintenance system, although other comparable domains are acceptable.

Regardless of the specific topic, the system is expected to model meaningful real-world entities and the relationships between them. Your program should therefore reflect how information is naturally organized and used in that setting, rather than storing unrelated values or acting as a collection of isolated functions.

3. Initial Data and Persistence

Your program should behave like a real application rather than a short-lived script. When the system runs for the first time, it must initialize its own working environment by generating a dataset and saving it to file. From that point onwards, the program must load and update this stored data instead of recreating it each time. The stored data should grow and change as the system is used.

The dataset must:

- contain a substantial number of records
- include relationships between records
- persist between program executions
- evolve as users interact with the system

Hard-coded small example datasets or regenerating fresh data every run are **not acceptable**.

4. Functional Requirements

Your system should provide a coherent set of operations appropriate to your chosen domain.
The user should be able to interact with stored records in a meaningful way rather than simply viewing static information.

The system must support:

- creating records
- updating records
- deleting records

- searching and querying records
- saving and loading data
- handling incorrect input safely

The program should continue running whenever possible and must not terminate due to normal user mistakes.

5. Technical Requirements

5.1 Object-Oriented Design

The program should be structured as an object-oriented system with clear responsibilities assigned to classes. Relationships between classes should reflect the real-world domain you model. Your implementation must demonstrate:

- classes and objects
- encapsulation
- inheritance
- polymorphism
- composition relationships
- abstract or base classes
- custom exceptions

5.2 Data Structures

You are required to implement and use your own fundamental data structures. These structures should be integrated into the behavior of your system rather than implemented separately. Required structures:

- Doubly linked list
- Stack
- Queue
- Tree
- Binary search tree
- Hash table / map structure

5.3 Algorithms and Recursion

Your system should demonstrate algorithmic reasoning rather than simple storage and retrieval. Your implementation must include:

- recursive tree traversal
- searching behavior analysis
- consideration of efficiency and complexity

5.4 File Storage

The system must store its data in files and restore the previous state when restarted.
Users should be able to stop and restart the program without losing information.

5.5 Testing

Testing should guide development rather than be performed only at the end. You must include:

- unit testing (individual methods)
- class/module testing
- system testing

Testing should be used to discover faults and improve the design.

Final Report Submission Format

You should supply the following items as **one zip file** for your team.

One member of the group should upload the submission on behalf of everyone.
Please ensure that all group member's names appear clearly in the report and presentation material, and that **individual contributions are explained in the first page of the report** and the video.

Main Written Report (maximum 10 pages)

The main report should explain the overall design and development of your system. It should describe what your system is intended to do, how it is structured, and why you designed it in this way. In particular, you should explain:

- how the application models the chosen domain
- how your design evolved during development
- what problems you encountered and why changes were made
- how your system provides useful functionality to its users
- how testing and debugging informed your decisions

Your report should cover the following four areas:

1. Problem and System Design

- what real-world situation your system represents and what problem it helps to manage
- the main entities in your domain and how they relate to each other
- the overall structure of the program and how the user interacts with it
- why you organized the system in this way rather than a simpler script-based solution

Your explanation should make it clear that the program is designed as a coherent system rather than a collection of separate features.

2. Technical Design

- how the object-oriented structure was designed and how responsibilities are divided between classes
- how inheritance, composition, and abstraction were used to organize behavior
- where and why each data structure is used in the program
- important algorithms or operations and how efficiency influenced your design decisions

Focus on justification: describe why these choices make sense for your specific system.

3. Development and Testing

- how the system changed during development and what caused those changes
- at least one significant problem you encountered and how you diagnosed it
- how testing was carried out and what you learned from it
- how debugging or testing led to improvements in the design

This section should show that the system was refined over time rather than written once and left unchanged.

4. Reflection

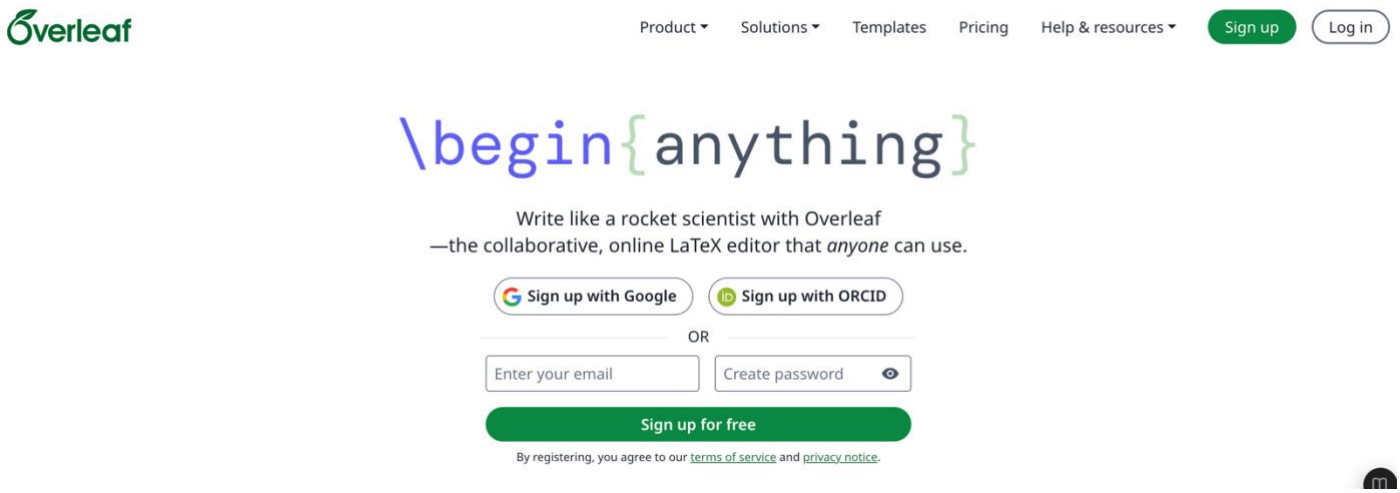
- what aspects of the system work well and why
- the limitations of your current design
- what you would change if you continued development
- what you learned about software design from building the project

The reflection should focus on understanding and reasoning rather than listing features.

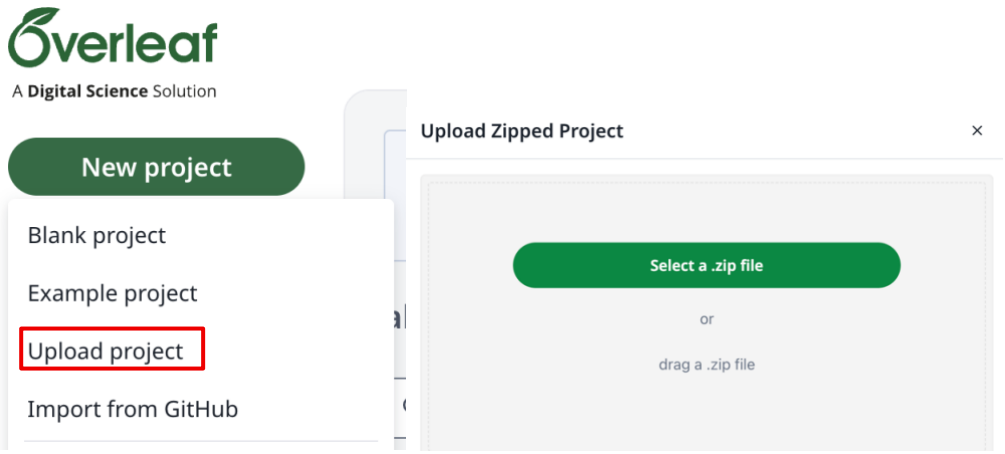
Report Format and Template

The final report should use Latex template to finish. The Latex template file can be downloaded from MyAberdeen. Here is a suggestion about how to use overleaf, an online Latex platform, to prepare your group report:

1. **Register an overleaf account:** open https://www.overleaf.com/ and register an account following the Figure below



2. **Build a Latex project:** Click “New project” and choose “Upload project” to upload the **latex_template.zip** downloaded from MyAberdeen Assessment 2 folder following the Figure below



Important dates for group project

1. Build a project group (4-6 persons) on MyAberdeen, Deadline: March 16, 2026 at 22:00, SCNU time
2. Deadline for final report + code + video submission: May 8, 2026 at 22:00, SCNU time

Marks breakdown for group project

The group project will be assessed using three components:

1. Written Report — 80%

The main component of the assessment is the written report. This evaluates the technical depth and understanding of the work carried out by the group. The report should clearly explain:

- system design and architecture
- object-oriented modelling decisions
- implementation of data structures
- algorithm design and complexity considerations
- testing strategy and fault detection
- debugging and development evolution
- justification of design choices and trade-offs

2. Demonstration Video (up to 10 mins) — 10%

A short demonstration video showing the software running. The video must show real execution of the program and demonstrate:

- creation of data
- searching and updating records
- deletion operations
- handling of invalid input

3. Process Presentation Video (up to 10 mins) — 10%

A short presentation explaining the development process of the project. The group should explain:

- how the system evolved over time
- key problems encountered
- major design changes and why they were necessary
- how testing influenced the implementation

In addition, the video must clearly identify the contribution of each group member. It must briefly explain each student own work, responsibilities, and technical decisions they personally implemented. This component assesses understanding of the development process and individual contribution.